AI-Powered Real Estate Property Triage System

Final Project Book — AI Engineering Course

Author: Yehuda Rokach **Project type:** Individual final project **Date:** June 2026 **Status:** Complete and deployed — WebUI + 4 microservices + n8n orchestration live on a GPU EC2 instance (`g4dn.xlarge` / NVIDIA Tesla T4), verified end-to-end; image classifier at 84.6% room-type accuracy; assistant chat grounded on the Bedrock Knowledge Base.

Table of Contents

1. Abstract

This project designs and builds a production-style, multi-modal AI system that automates the intake and initial evaluation of real-estate property listings for a fictional agency. A listing agent submits a free-text description plus property photos; the system validates the submission, extracts structured fields, classifies each image by room type and condition, retrieves comparable past listings, and produces a ready-to-publish listing brief — then routes it to the correct team.

The system is assembled from managed and reusable building blocks rather than hand-rolled from scratch: **Amazon Bedrock Knowledge Bases** for retrieval, **Amazon Bedrock Guardrails** for safety, **Google Gemini** for all LLM generation, a **PyTorch** transfer-learning model for image analysis, **n8n** for orchestration, and a **Flask + Ollama** web interface.

The full system is **deployed and running on AWS EC2** (a `g4dn.xlarge` GPU instance) and verified end-to-end: the WebUI, all four microservices (RAG, Image Analyser, Guardrails, LangGraph Agent) — covered by a 43-test offline suite — and the **n8n 8-node orchestration flow**. The image classifier reaches **84.4%** room-type accuracy and adds a **trained condition-score head** (1–5); the full pipeline passes both guardrails with correct residential/commercial routing; and the assistant chat is **grounded on the Bedrock Knowledge Base** and runs on the GPU (~2–3 s per response). Submitted listings and dashboard events persist in **Amazon DynamoDB** (a durable system-of-record that survives container rebuilds), and uploaded photos are served from S3 and surfaced in both the dashboard and the chat.

2. Introduction & Real-World Scenario

A real-estate agency receives dozens of new property submissions every day, each a written description plus photographs. Staff must check the submission is genuine (not spam/off-topic), identify property type/condition/features, score the uploaded images, find similar past listings, route the listing to residential vs. commercial teams, and produce a clean published brief.

This is a realistic, multi-modal workflow: it combines text understanding, image analysis, retrieval-augmented generation, safety filtering, and agent-based reasoning in one coherent product. This project automates the entire pipeline end-to-end.

Learning objectives demonstrated: deploying a multi-service AI system on cloud infrastructure; building n8n automation flows with AI nodes; a retrieval-augmented pipeline; a PyTorch image classifier; prompt engineering across six distinct surfaces; input/output safety guardrails; a multi-step reasoning agent; and a local-LLM conversational UI.

3. System Architecture

The system has four layers, each communicating with the next over HTTP.

Layer 1 — Web UI (HTML/CSS/JS + Flask). Two working surfaces plus a monitoring extension: (a) a conversational assistant backed by an **Ollama** Llama 3.1 server (on the instance GPU), **grounded on the Bedrock Knowledge Base** — each turn builds its retrieval query from the recent conversation and pins any listing already discussed, so follow-up questions stay coherent, and it answers by stable ID; photos of a referenced listing appear beneath the reply; (b) a listing submission form that uploads photos to **S3**, runs them through the Image Analyser, POSTs to the n8n webhook (behind a full-screen pipeline loader), and renders the brief plus a per-photo room/condition grid; (c) a monitoring dashboard (Chart.js) whose rows open a per-listing detail view. Built as a custom Flask app (instructor permits) for full design control.

Layer 2 — n8n orchestration. An 8-node flow: webhook trigger → guardrails input check → IF (pass/fail) → Information Extractor (Gemini) → AI Agent (Gemini; dispatches tool calls to the services — it always invokes the **LangGraph agent**, which itself orchestrates RAG + Image, and may call `rag_lookup`/`image_analyser` directly) → LLM Chain (final brief) → guardrails output check → router (residential vs. commercial).

Layer 3 — FastAPI microservices. Four independent, containerised services: RAG, Image Analyser, Guardrails, and a LangGraph Agent. Each exposes a single well-defined endpoint.

Layer 4 — Managed services & external LLM. Amazon Bedrock Knowledge Bases (Titan V2 embeddings on **S3 Vectors**) and Amazon Bedrock Guardrails; **Amazon DynamoDB** as the durable system-of-record for submitted listings and dashboard events (so data survives container rebuilds); **Amazon S3** for uploaded photos; Google Gemini for all text generation (including the served image condition score); Ollama (local) for the assistant chat.

A full architecture diagram and the intentional deviations from the guideline are in `docs/architecture.md`.

4. Technology Choices

This project intentionally **assembles managed services** instead of hand-rolling each component, which is both the instructor's intent (a few hours of assembly) and good engineering for a solo build.

Two AWS Bedrock services (requirement: ≥2). - **Bedrock Knowledge Bases** powers the RAG service — managed vector store, embeddings, and retrieval, reusing a working pattern the author already built. - **Bedrock Guardrails** powers the Guardrails service — managed content filters, PII handling, and contextual grounding.

Gemini for all LLM generation. Rather than adding a third Bedrock dependency, all generation (RAG insight, guardrail classification/auditing, the agent's reasoning, and the n8n nodes) uses Google Gemini, which the author already uses fluently. The Gemini API key is stored in **AWS Secrets Manager** and pulled by each service using its existing AWS credentials — no secret in the repo.

Intentional deviations from the written guideline. The guideline lists NeMo Guardrails, Llama.cpp, and a local ChromaDB. We substitute **Bedrock Guardrails** (for NeMo) and **Bedrock Knowledge Bases** (for Llama.cpp + ChromaDB). This satisfies the ≥2-Bedrock-services requirement, reduces operational burden for a solo developer, and — as a side effect — already satisfies the "managed vector store" optional extension.

PyTorch transfer learning (EfficientNet-B0) for the Image Analyser, because the rubric explicitly rewards a trained model.

Component	Technology
Web UI	HTML/CSS/JS + Flask
Conversational LLM	Ollama + Llama 3.1 (local)
Orchestration	n8n
LLM generation	Google Gemini
RAG	Bedrock Knowledge Base (Titan V2 on S3 Vectors)
Image Analyser	PyTorch + EfficientNet-B0, two heads (room 7-class incl. `not_a_room`; condition 1–5). Condition served by Gemini Vision
Guardrails	Bedrock Guardrails (`ApplyGuardrail`) + Gemini
Agent	LangGraph + Gemini
Storage (system-of-record)	Amazon DynamoDB (listings + events); Amazon S3 (photos)
Secrets	AWS Secrets Manager

5. Component Design

5.1 Web UI (built first)

A 3-tab app served by **Flask** with a vanilla HTML/CSS/JS frontend. **Assistant** tab streams chat from the Ollama `/api/chat` endpoint (GPU), **grounded on the RAG service / Bedrock KB** — it builds its retrieval query from the recent conversation (so a vague follow-up keeps the topic) and pins any listing already named in the chat, answers by stable ID, refuses off-topic questions, and

invents nothing (prompt surface #5, V8); photos of a referenced listing are shown beneath the reply (click → detail). **Submit Listing** tab uploads photos to **S3**, calls the **Image Analyser** on each, posts the description + agent name to the n8n webhook (behind a full-screen pipeline loader), and renders the brief + a per-photo room/condition grid; **accepted listings are ingested back into the KB** so they become comparables and the chat can answer about them. **Dashboard** tab shows live stats (Chart.js); each row opens a per-listing detail (description, brief, photos). Submitted listings and events persist in **Amazon DynamoDB** (durable across container rebuilds); photos are served from S3 by permanent URL. During early development the submit tab is tested against a mock brief so the whole UI is demoable before n8n exists.

5.2 RAG service (`POST /query`)

Embeds the description and retrieves the top-3 most similar past listings from a Bedrock Knowledge Base pre-populated with ≥20 synthetic listings, then generates a short insight with Gemini that cites the listing it drew from and never fabricates facts. Output: `{ similar_listings, insight }` (a lightweight `retrieve-only` mode skips the insight — used by the assistant chat). **Accepted submissions are ingested back into the KB** (text written to the S3 data source + an ingestion job), so each new listing becomes a retrievable comparable for future queries and for the chat.

5.3 Image Analyser (`POST /analyse`)

A transfer-learning EfficientNet-B0 (ImageNet weights, frozen backbone) with **two heads on a shared backbone**: a room-type head over **7 classes** (kitchen, bathroom, living room, bedroom, exterior, other, and a `not_a_room` reject class added beyond the spec so non-property photos are flagged rather than forced into a room) and a **condition-score head (1–5)** — the spec's "second output head". Below a 0.55 confidence threshold the room is `uncertain`. Output: `{ room_type, condition_score, confidence }`. Room labels come from public Kaggle datasets via `kagglehub` (500/class; rooms from *robinreni/house-rooms*, exterior from *mikhailma* street data, negatives from *prasunroy/natural-images*). Condition labels — for which no public ground truth exists — were **bootstrapped with Gemini Vision** over 846 images (clean rooms plus lower-condition sources for range: *messy-vs-clean-room* and real apartment photos from *home-bro-images*), then distilled into the head with inverse-frequency class weights. **Serving is a hybrid:** room type comes from the CNN; the **condition score is served by Gemini Vision** because the trained head, limited by available data, was unreliable on degraded-but-tidy rooms — the head satisfies the spec and is the offline fallback. Full evaluation in `docs/model_card.md`.

5.4 Guardrails service (`POST /check/input`, `POST /check/output`)

Input check: Bedrock `ApplyGuardrail` for safety (content filters, denied topics, profanity) plus a Gemini classifier that accepts only genuine property listings in the expected language (rejecting others, including unexpected languages — the multilingual extension). Output check: Bedrock safety plus a Gemini factuality-vs-source check that catches invented prices, fabricated certifications, and false legal claims. Output: `{ pass, reason, safe_text }`. (PII masking was

removed — the system has no email/phone fields — and `_apply_guardrail` fails *closed* on any intervention.)

5.5 LangGraph Agent (`POST /agent/run`)

A 3-node state graph — planner → tool executor → synthesiser — using Gemini, where the executor calls the RAG and Image Analyser services. Answers multi-step questions (e.g., "what renovation work would bring this property to condition score 5?"). Output: `{ answer, tools_used, reasoning_steps }`.

5.6 n8n flow

Eight nodes wiring the webhook through the guardrails, the Gemini Information Extractor and AI Agent, the LLM Chain that produces the brief, the output guardrail, and a router that sends residential vs. commercial listings to different teams. The AI Agent's prompt mandates a call to the **LangGraph agent** on every listing (Surface #2 V2), so the multi-step planner→tools→synthesiser reasoner is part of the normal pipeline, not only a standalone endpoint.

6. Prompt Engineering

Seven prompt surfaces are tuned and logged with ≥10 test cases and a measured pass rate per surface (the guideline asks for five): the n8n Information Extractor, the n8n AI Agent prompt, the RAG insight/citation prompt, the Guardrails rail prompts, the Ollama system prompt, the LangGraph Agent tool descriptions, and the image condition rubric (Gemini Vision). The full iteration history is in docs/prompt_log.md.

Results: #1 Extractor 9/10 (0 inventions) · #2 n8n Agent 10/10 + brief V1→V4 · #3 RAG insight V1 = 10/10, **0 fabricated listing ids** · #4 Guardrails V1 = 11/11, 0% false positives · #5 Ollama V1→V8 = 10/10 (key fixes: forbidding invented URLs/prices, anti-prompt-injection, in-code language matching, listings-awareness, V7 RAG-grounding with stable-ID references, and **V8: conversation-context retrieval + pinning of already-discussed listings so a vague follow-up like "tell me about it" no longer drops the listing and contradicts an earlier turn**) · #6 Agent tool descriptions 9/10 routing · #7 image condition rubric V1→V2 = 9/10 directional (used to both bootstrap the training labels and serve the live score).

7. Results & Evaluation

Image Analyser. Room-type validation accuracy (dual-head model): **84.4%** — clears the >75% bar; the `not_a_room` reject class correctly flags dogs, cats, documents and diagrams where a 6-class model was overconfidently wrong. The **condition head** reaches validation MAE ≈ 0.6 (within ~½ a point of the Gemini-bootstrapped label on a 1–5 scale), but on out-of-distribution "bad room" photos it was unreliable — it learned *messy/dirty → low* yet mis-scored degraded-but-tidy rooms — so the **served condition score comes from Gemini Vision** (reliable across condition

types), with the trained head as the spec deliverable and offline fallback. Full confusion matrix, condition rubric and OOD analysis in `docs/model_card.md`.

Guardrails. Surface #4 V1: 11/11 correct decisions, 0% false positives on the valid-listing set; spam/off-topic/offensive all blocked; non-Hebrew/English rejected with a localized message.

RAG. Surface #3 V1: 10/10, 0 fabricated listing ids (regex-verified), citations in every insight.

Testing. A 43-test offline `pytest` suite (AWS/Gemini/Ollama mocked) covers the shared helpers, all four services, and the WebUI — re-runnable with no credentials or network.

Still to do: RAG precision@3 benchmark (managed-vector-store extension), and an end-to-end run with screenshots once n8n is wired.

8. Deployment Notes

The full stack — the four FastAPI services, n8n, Ollama, and the WebUI — runs on a single **AWS EC2** instance via Docker Compose, as a **public site on the internet**, verified end-to-end.

- **Instance:** `g4dn.xlarge` with an **NVIDIA Tesla T4 GPU**, Amazon Linux 2023, `us-east-1`, gp3 root. It launched as a CPU instance (`t3.xlarge`) and was **resized to the GPU type once the on-demand-G vCPU quota was approved** — the EBS volume, Elastic IP, and data all carried over.
- **GPU for the assistant:** the NVIDIA open-kernel-module driver (built via DKMS) plus `nvidia-container-toolkit` are installed, and the `ollama` container is granted the GPU — cutting assistant responses from **~57 s on CPU to ~2–3 s** on the T4.
- **Bootstrap** (`deploy/ec2-userdata.sh`): installs Docker + the Compose plugin, git-clones the repo, pulls the trained `model.pth` from S3, and runs `docker compose up --build -d`.
- **Public access:** a stable **Elastic IP**; the security group exposes the WebUI (`:5050`) and n8n (`:5678`). Management is still done through **AWS Systems Manager** (no SSH key).
- **Credentials — no keys on the box:** the instance carries an **IAM role**; services read the Gemini key from **Secrets Manager** and call **Bedrock** (KB `Retrieve`, `ApplyGuardrail`, `StartIngestionJob`), **S3** (`GetObject`/`PutObject` for photo uploads + KB ingestion), and **DynamoDB** (`PutItem`/`GetItem`/`Scan` on the two tables) through it.
- **n8n hosted on the box:** the orchestration flow runs in the same Compose stack; sibling services are reached via the host gateway, the Gemini credential is configured once, and the workflow + credentials persist in a Docker volume.
- **Persistence (durable system-of-record):** submitted listings (`pt_listings`) and dashboard events (`pt_events`) are stored in **Amazon DynamoDB** — they survive container rebuilds and stop/start, which an earlier Docker-volume store did not. Uploaded photos live in **S3** under a public-read `uploads/` prefix, so their URLs are permanent (no presigning to expire). The Knowledge Base (S3 + S3 Vectors) is durable by design.

- **Cost control:** `g4dn.xlarge` (~$0.53/hr) is **stopped when idle**; the GPU driver, volumes, and Elastic IP persist across stop/start, and containers auto-resume via `restart: unless-stopped`.
- **Docker on Apple Silicon:** build `--platform linux/amd64` for parity with the x86 EC2 host.

9. Conclusions & Future Work

To be written at the end. Candidate future work: the human-in-the-loop feedback/active-learning loop (deferred), a managed Pinecone/Weaviate comparison, and richer monitoring.

10. References

- Project guideline: *AI Engineering — Final Project: AI-Powered Real Estate Property Triage System.*
- Amazon Bedrock Knowledge Bases, Amazon Bedrock Guardrails (AWS documentation).
- Google Gemini API documentation.
- PyTorch transfer-learning tutorial; torchvision model zoo.
- n8n documentation; Ollama documentation; NVIDIA CUDA / container-toolkit documentation.